

Quy hoạch động (Dynamic Programming)

Giảng viên: Trần Đình Tân

Môn: Thuật toán ứng dụng

Sau bài học, người học có thể:

- Nhận biết bài toán có **tính tối ưu con** và **bài toán con chồng lấp**
- Thiết kế công thức truy hồi và bảng quy hoạch động
- Phân biệt hai cách cài đặt: *memoization* và *tabulation*
- Áp dụng quy hoạch động vào các bài toán kinh điển: Fibonacci, đổi tiền, ba lô 0-1 và LCS
- Phân tích đúng độ phức tạp thời gian và bộ nhớ của thuật toán

Nội dung

- 1 Ý tưởng và nguyên lý cốt lõi
- 2 Hai cách cài đặt
- 3 Bài toán đổi tiền
- 4 Bài toán ba lô 0-1
- 5 Bài toán dãy con chung dài nhất
- 6 Mẫu bài toán và lưu ý
- 7 Tổng kết

Quy hoạch động là gì?

Định nghĩa

Quy hoạch động (dynamic programming) là kỹ thuật giải bài toán bằng cách chia thành các bài toán con, lưu kết quả của chúng, rồi tái sử dụng để tránh tính lặp lại.

Trực giác: Nếu cùng một câu hỏi nhỏ xuất hiện nhiều lần, ta chỉ trả lời một lần và ghi lại kết quả.

Hai điều kiện thường gặp:

- 1 **Tính tối ưu con** (Optimal substructure)
- 2 **Bài toán con chồng lấp** (Overlapping subproblems)

Vì sao cần quy hoạch động?

Xét Fibonacci:

$$F(n) = F(n-1) + F(n-2), \quad F(0) = 0, F(1) = 1.$$

Đệ quy trực tiếp

- Tính lại cùng một giá trị nhiều lần
- Ví dụ $F(4)$ xuất hiện trong cả $F(5)$ và $F(6)$
- Thời gian tăng theo hàm mũ

Quy hoạch động

- Mỗi $F(i)$ chỉ tính một lần
- Lưu trong mảng hoặc biến tạm
- Thời gian giảm còn tuyến tính

Ý tưởng chính

Đổi thời gian tính lại lấy bộ nhớ lưu trữ.

Tính tối ưu con

Định nghĩa

Một bài toán có **tính tối ưu con** nếu nghiệm tối ưu của bài toán lớn có thể được xây dựng từ nghiệm tối ưu của các bài toán con.

Ví dụ — đường đi ngắn nhất:

Nếu đường đi ngắn nhất từ s đến t đi qua v , thì đoạn từ s đến v cũng phải là đường đi ngắn nhất từ s đến v .

- Nếu đoạn con không tối ưu, ta có thể thay bằng đoạn tốt hơn.
- Khi đó đường đi từ s đến t cũng tốt hơn, mâu thuẫn với tối ưu.
- Đây là kiểu lập luận thường dùng khi chứng minh công thức QHD.

Bài toán con chồng lấp

Định nghĩa

Các bài toán con **chồng lấp** khi trong quá trình giải, cùng một bài toán con được gọi hoặc cần dùng nhiều lần.

Fibonacci:

$$F(6) = F(5) + F(4), \quad F(5) = F(4) + F(3).$$

Giá trị $F(4)$ được cần nhiều lần. Với n lớn, số lần lặp lại tăng rất nhanh.

Dấu hiệu nhận biết

Nếu cây đệ quy có nhiều nhánh dẫn đến cùng một trạng thái, hãy nghĩ đến quy hoạch động.

So sánh ba kỹ thuật

Chia để trị

- Bài toán con độc lập
- Không cần lưu nhiều
- VD: Merge Sort

Tham lam

- Chọn tốt nhất cục bộ
- Không xét lại
- VD: Huffman

Quy hoạch động

- Bài toán con chồng lấp
- Lưu kết quả trung gian
- VD: LCS, ba lô 0-1

Câu hỏi chọn phương pháp

Có cần thử nhiều lựa chọn và các trạng thái lặp lại không? Nếu có, QHĐ thường là ứng viên tốt.

Quy trình thiết kế quy hoạch động

- 1 **Xác định trạng thái:** $dp[...]$ đại diện cho điều gì?
- 2 **Xác định đáp án:** đáp án nằm ở ô nào của bảng?
- 3 **Viết truy hồi:** trạng thái hiện tại phụ thuộc trạng thái nào?
- 4 **Đặt điều kiện cơ sở:** giá trị ban đầu khi bài toán rất nhỏ
- 5 **Chọn thứ tự tính:** bảo đảm trạng thái phụ thuộc đã có trước
- 6 **Phân tích độ phức tạp:** số trạng thái $\times$ chi phí mỗi trạng thái

Mẫu tư duy

Độ phức tạp = số trạng thái $\times$ số lựa chọn cho mỗi trạng thái.

Memoization: từ trên xuống

Ý tưởng

Viết đệ quy tự nhiên, nhưng trước khi tính một trạng thái, kiểm tra xem trạng thái đó đã được lưu trong bảng nhớ chưa.

- Dễ chuyển từ công thức truy hồi sang code
- Chỉ tính các trạng thái thật sự cần dùng
- Cần chú ý độ sâu đệ quy và bộ nhớ stack

Mẫu:

$$\text{solve}(x) = \begin{cases} \text{giá trị cơ sở,} & x \text{ nhỏ} \\ \text{memo}[x], & x \text{ đã tính} \\ \text{tính từ các } \text{solve}(y), & \text{ngược lại} \end{cases}$$

Thuật toán Fibonacci bằng memoization

Algorithm 1 FIB-MEMO(n)

- 1: Khởi tạo mảng $memo[0..n]$ với giá trị rỗng
 - 2: $memo[0] \leftarrow 0, memo[1] \leftarrow 1$
 - 3: **return** FIB-AUX($n, memo$)
-

Algorithm 2 FIB-AUX($i, memo$)

- 1: **if** $memo[i]$ đã có giá trị **then**
 - 2: **return** $memo[i]$
 - 3: **end if**
 - 4: $memo[i] \leftarrow \text{FIB-AUX}(i - 1, memo) + \text{FIB-AUX}(i - 2, memo)$
 - 5: **return** $memo[i]$
-

Tabulation: từ dưới lên

Ý tưởng

Tính các trạng thái nhỏ trước, sau đó dùng chúng để tính các trạng thái lớn hơn theo một thứ tự xác định.

- Không cần đệ quy
- Dễ kiểm soát thứ tự tính và bộ nhớ
- Thường phù hợp khi cần điền toàn bộ bảng

Fibonacci:

$$dp[0] = 0, \quad dp[1] = 1, \quad dp[i] = dp[i - 1] + dp[i - 2] \quad (2 \leq i \leq n).$$

Thuật toán Fibonacci từ dưới lên

Algorithm 3 FIB-TABULATION(n)

```
1: if  $n \leq 1$  then
2:   return  $n$ 
3: end if
4:  $dp[0] \leftarrow 0, dp[1] \leftarrow 1$ 
5: for  $i = 2$  to  $n$  do
6:    $dp[i] \leftarrow dp[i - 1] + dp[i - 2]$ 
7: end for
8: return  $dp[n]$ 
```

- Số trạng thái: $n + 1$
- Chi phí mỗi trạng thái: $O(1)$
- Thời gian: $O(n)$, bộ nhớ: $O(n)$

Tối ưu bộ nhớ

Với Fibonacci, $dp[i]$ chỉ phụ thuộc hai trạng thái trước:

$$dp[i] = dp[i - 1] + dp[i - 2].$$

Vì vậy không cần lưu toàn bộ mảng.

Algorithm 4 FIB-SPACE-OPT(n)

```
1: if  $n \leq 1$  then  
2:   return  $n$   
3: end if  
4:  $a \leftarrow 0, b \leftarrow 1$   
5: for  $i = 2$  to  $n$  do  
6:    $c \leftarrow a + b$   
7:    $a \leftarrow b, b \leftarrow c$   
8: end for  
9: return  $b$ 
```

Thời gian $O(n)$, bộ nhớ $O(1)$.

Memoization và tabulation

Tiêu chí	Memoization	Tabulation
Hướng tính	Trên xuống	Dưới lên
Cài đặt	Đệ quy + bảng nhớ	Vòng lặp + bảng
Trạng thái tính	Khi cần	Theo thứ tự định trước
Rủi ro	Stack sâu	Sai thứ tự điền bảng
Phù hợp	Truy hồi tự nhiên	Bảng nhỏ, thứ tự rõ

Kết luận

Hai cách có cùng bản chất: mỗi trạng thái chỉ được tính một lần.

Đổi tiền: số đồng xu ít nhất

Bài toán

Cho các mệnh giá $D = \{d_1, d_2, \dots, d_k\}$ và số tiền n . Tìm số đồng xu ít nhất có tổng bằng n .

Khác bài tham lam: Với mệnh giá tùy ý, chọn đồng lớn nhất trước không luôn tối ưu.

Ví dụ:

- $D = \{4, 3, 1\}$, $n = 6$
- Tham lam: $4 + 1 + 1$ dùng 3 đồng
- Tối ưu: $3 + 3$ dùng 2 đồng

Trạng thái và công thức truy hồi

Trạng thái

$dp[x]$ = số đồng xu ít nhất để đổi được số tiền x .

Điều kiện cơ sở:

$$dp[0] = 0.$$

Truy hồi:

$$dp[x] = 1 + \min_{\substack{d \in D \\ d \leq x}} dp[x - d].$$

- Chọn đồng xu cuối cùng có mệnh giá d
- Trước đó cần đổi tối ưu số tiền $x - d$
- Thử tất cả mệnh giá có thể làm đồng xu cuối cùng

Thuật toán đổi tiền bằng quy hoạch động

Algorithm 5 MIN-COINS(n, D)

```
1:  $dp[0] \leftarrow 0$ 
2: for  $x = 1$  to  $n$  do
3:    $dp[x] \leftarrow +\infty$ 
4:   for mỗi mệnh giá  $d \in D$  do
5:     if  $d \leq x$  then
6:        $dp[x] \leftarrow \min(dp[x], 1 + dp[x - d])$ 
7:     end if
8:   end for
9: end for
10: return  $dp[n]$ 
```

Số trạng thái: $n + 1$, mỗi trạng thái thử k mệnh giá.

Thời gian $O(nk)$, bộ nhớ $O(n)$.

Ví dụ bảng đổi tiền

Với $D = \{1, 3, 4\}$, tính $dp[x]$ cho $x = 0, \dots, 6$:

x	0	1	2	3	4	5	6
$dp[x]$	0	1	2	1	1	2	2

Diễn giải:

- $dp[3] = 1$ vì dùng một đồng 3
- $dp[4] = 1$ vì dùng một đồng 4
- $dp[6] = 1 + \min(dp[5], dp[3], dp[2]) = 1 + \min(2, 1, 2) = 2$
- Nghiệm tối ưu cho 6 là $3 + 3$

Bài toán

Có n đồ vật. Vật i có trọng lượng w_i và giá trị v_i . Ba lô có sức chứa W . Mỗi vật được chọn hoặc không chọn. Tối đa hóa tổng giá trị mà không vượt quá sức chứa.

Điểm khác ba lô phân số:

- Không được chia nhỏ vật
- Tiêu chí tham lam theo v_i/w_i có thể sai
- Cần thử hai khả năng: **chọn** hoặc **không chọn**

Trạng thái ba lô 0-1

Trạng thái

$dp[i][c]$ = giá trị lớn nhất có thể đạt được khi xét i vật đầu tiên và sức chứa còn xét là c .

Điều kiện cơ sở:

$$dp[0][c] = 0, \quad dp[i][0] = 0.$$

Đáp án:

$$dp[n][W].$$

Trạng thái có hai chiều vì quyết định phụ thuộc vào:

- Đã xét bao nhiêu vật
- Sức chứa còn lại là bao nhiêu

Công thức truy hồi ba lô

Khi xét vật i :

$$dp[i][c] = \begin{cases} dp[i-1][c], & w_i > c, \\ \max\{dp[i-1][c], dp[i-1][c - w_i] + v_i\}, & w_i \leq c. \end{cases}$$

Ý nghĩa hai lựa chọn:

- **Không chọn vật i :** giữ giá trị tốt nhất từ $i - 1$ vật
- **Chọn vật i :** nhận thêm v_i , còn sức chứa $c - w_i$
- Lấy lựa chọn có giá trị lớn hơn

Algorithm 6 ZERO-ONE-KNAPSACK(w, v, n, W)

```
1: Khởi tạo  $dp[0][c] \leftarrow 0$  với mọi  $0 \leq c \leq W$ 
2: for  $i = 1$  to  $n$  do
3:   for  $c = 0$  to  $W$  do
4:      $dp[i][c] \leftarrow dp[i-1][c]$ 
5:     if  $w_i \leq c$  then
6:        $dp[i][c] \leftarrow \max(dp[i][c], dp[i-1][c - w_i] + v_i)$ 
7:     end if
8:   end for
9: end for
10: return  $dp[n][W]$ 
```

Thời gian $O(nW)$, bộ nhớ $O(nW)$.

Ví dụ ba lô 0-1

Cho $W = 5$ và ba vật:

Vật	1	2	3
w_i	2	3	4
v_i	3	4	5

Một vài trạng thái:

- Với vật 1: $dp[1][2] = 3$, $dp[1][5] = 3$
- Với vật 2 và sức chứa 5: $dp[2][5] = \max(dp[1][5], dp[1][2] + 4) = \max(3, 7) = 7$
- Với vật 3 và sức chứa 5: $dp[3][5] = \max(dp[2][5], dp[2][1] + 5) = \max(7, 5) = 7$

Đáp án: chọn vật 1 và 2, tổng giá trị **7**.

Tối ưu bộ nhớ cho ba lô

Vì $dp[i][c]$ chỉ phụ thuộc hàng $i - 1$, có thể dùng mảng một chiều:

$$dp[c] = \max(dp[c], dp[c - w_i] + v_i).$$

Algorithm 7 ZERO-ONE-KNAPSACK-1D(w, v, n, W)

```
1: Khởi tạo  $dp[0..W] \leftarrow 0$ 
2: for  $i = 1$  to  $n$  do
3:   for  $c = W$  downto  $w_i$  do
4:      $dp[c] \leftarrow \max(dp[c], dp[c - w_i] + v_i)$ 
5:   end for
6: end for
7: return  $dp[W]$ 
```

Lưu ý

Vòng lặp sức chứa phải đi **giảm dần** để mỗi vật chỉ được dùng một lần.

Bài toán

Cho hai chuỗi $X = x_1x_2 \dots x_m$ và $Y = y_1y_2 \dots y_n$. Tìm độ dài **dãy con chung dài nhất** (Longest Common Subsequence).

Dãy con giữ nguyên thứ tự, nhưng không cần liên tiếp.

Ví dụ:

- $X = \text{ABCB DAB}$
- $Y = \text{BDCABA}$
- Một LCS là BCBA, độ dài 4

Trạng thái LCS

Trạng thái

$dp[i][j]$ = độ dài LCS của hai tiền tố $X[1..i]$ và $Y[1..j]$.

Điều kiện cơ sở:

$$dp[0][j] = 0, \quad dp[i][0] = 0.$$

Đáp án:

$$dp[m][n].$$

Ta xét ký tự cuối của hai tiền tố: x_i và y_j .

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1, & x_i = y_j, \\ \max\{dp[i-1][j], dp[i][j-1]\}, & x_i \neq y_j. \end{cases}$$

Diễn giải:

- Nếu $x_i = y_j$, ký tự này có thể ghép vào LCS của hai tiền tố trước
- Nếu khác nhau, bỏ một trong hai ký tự cuối rồi lấy phương án tốt hơn
- Mỗi ô bảng phụ thuộc ô trên, ô trái và ô chéo trên-trái

Algorithm 8 LCS-LENGTH(X, Y)

```
1:  $m \leftarrow |X|, n \leftarrow |Y|$ 
2: Khởi tạo  $dp[i][0] \leftarrow 0, dp[0][j] \leftarrow 0$ 
3: for  $i = 1$  to  $m$  do
4:   for  $j = 1$  to  $n$  do
5:     if  $X[i] = Y[j]$  then
6:        $dp[i][j] \leftarrow dp[i-1][j-1] + 1$ 
7:     else
8:        $dp[i][j] \leftarrow \max(dp[i-1][j], dp[i][j-1])$ 
9:     end if
10:  end for
11: end for
12: return  $dp[m][n]$ 
```

Ví dụ bảng LCS

Với $X = ABC$ và $Y = AC$:

$dp[i][j]$	$\emptyset$	A	C
$\emptyset$	0	0	0
A	0	1	1
B	0	1	1
C	0	1	2

- Ô (1, 1) tăng lên 1 vì A trùng A
- Ô (2, 2) giữ 1 vì B khác C
- Ô (3, 2) tăng lên 2 vì C trùng C

Đáp án: LCS có độ dài **2**, là "AC".

Truy vết nghiệm LCS

Sau khi có bảng dp , có thể truy vết từ ô (m, n) :

- Nếu $x_i = y_j$, ký tự đó thuộc một LCS; ghi lại rồi đi chéo lên $(i - 1, j - 1)$
- Nếu $x_i \neq y_j$, đi về ô có giá trị lớn hơn: $(i - 1, j)$ hoặc $(i, j - 1)$
- Dừng khi $i = 0$ hoặc $j = 0$

Độ phức tạp

Tính độ dài: $O(mn)$ thời gian và $O(mn)$ bộ nhớ.

Truy vết một nghiệm: thêm $O(m + n)$ thời gian.

Các kiểu trạng thái thường gặp

Kiểu bài toán	Trạng thái thường dùng	Ví dụ
Một dãy	$dp[i]$	Fibonacci, LIS
Hai dãy	$dp[i][j]$	LCS, edit distance
Chọn/không chọn	$dp[i][c]$	Ba lô 0-1
Khoảng con	$dp[l][r]$	Nhân chuỗi ma trận
Đếm cách	$dp[x]$ hoặc $dp[i][x]$	Đổi tiền, đường đi lưới

Mẹo đặt trạng thái

Trạng thái nên chứa vừa đủ thông tin để quyết định bước tiếp theo, không thiếu và không thừa.

Những lỗi thường gặp

- **Đặt trạng thái mơ hồ:** không nói rõ $dp[i]$ đại diện cho gì
- **Thiếu điều kiện cơ sở:** bảng có ô chưa xác định
- **Sai thứ tự tính:** dùng trạng thái chưa được điền
- **Nhầm biến tối ưu:** tối đa hóa nhưng lại dùng min
- **Tối ưu bộ nhớ sai chiều:** ba lô 0-1 đi tăng dần sẽ dùng lại vật
- **Quên phân tích độ phức tạp:** không đếm số trạng thái và lựa chọn

Cách tự kiểm tra

Tính tay một ví dụ nhỏ và so sánh từng ô bảng với ý nghĩa trạng thái.

Bài tập trên lớp

- 1 Với $D = \{1, 4, 5\}$, lập bảng $dp[x]$ để đổi số tiền $x = 0, \dots, 8$. Kết quả tối ưu cho $x = 8$ là gì?
- 2 Cho ba lô $W = 6$ và các vật $(w, v) = \{(2, 4), (3, 5), (4, 7)\}$. Tính $dp[3][6]$.
- 3 Cho $X = \text{ABAZDC}$ và $Y = \text{BACBAD}$. Hãy tìm độ dài một LCS.
- 4 Chỉ ra trạng thái, truy hồi và điều kiện cơ sở cho bài toán: đếm số đường đi từ ô $(1, 1)$ đến ô (m, n) chỉ được đi xuống hoặc sang phải.

- Quy hoạch động phù hợp khi bài toán có tối ưu con và bài toán con chồng lấp
- Thiết kế QHD bắt đầu từ việc định nghĩa trạng thái rõ ràng
- Công thức truy hồi mô tả cách ghép nghiệm từ trạng thái nhỏ hơn
- Memoization và tabulation là hai cách cài đặt cùng một ý tưởng lưu kết quả
- Độ phức tạp thường được tính bằng số trạng thái nhân số lựa chọn
- Có thể tối ưu bộ nhớ khi trạng thái chỉ phụ thuộc một số lớp trước đó

- 1 Cài đặt MIN-COINS và in ra cả tập đồng xu được chọn.
- 2 Cài đặt ba lô 0-1 bản hai chiều và bản tối ưu bộ nhớ một chiều. So sánh kết quả trên cùng bộ dữ liệu.
- 3 Cài đặt LCS để in ra một dãy con chung dài nhất, không chỉ độ dài.
- 4 Tìm một bài toán thực tế có thể mô hình hóa bằng QHD. Nêu trạng thái, truy hồi, điều kiện cơ sở và độ phức tạp.

- Cormen, Leiserson, Rivest, Stein, *Introduction to Algorithms*, Chương 14–15.
- Kleinberg, Tardos, *Algorithm Design*, Chương 6.